预习作业：章节内容思维导图梳理

6.1 局部存储分配

1. 过程

- 含义
  - 过程定义是一个声明，它把一个名字和一段程序语句联系起来。
- 相关概念
  - 过程名：过程的名字。
  - 过程体：过程实际执行的语句部分。
  - 过程调用：执行被调用过程。
  - 形式参数：过程定义中的参数。
  - 实在参数：调用过程时传入的参数。
  - 函数：可以看作带返回值的过程。

2. 名字的作用域和绑定

- 声明
  - 声明是把某些信息与名字联系起来的语法结构。
  - 例如变量声明、过程声明、类型声明等。
- 作用域
  - 作用域是某个声明在程序中起作用的范围。
  - 如果一个名字出现在某个声明的作用域内，则该名字受该声明控制。
- 绑定
  - 绑定是指名字与具体存储单元或值建立联系。
  - 绑定通常发生在程序运行过程中。
- 环境与状态
  - 环境：名字到存储单元的映射。
  - 状态：存储单元到值的映射。
- 区别
  - 环境回答“名字对应哪个存储单元”。
  - 状态回答“这个存储单元中存放什么值”。

3. 静态概念与动态概念

静态概念	动态概念
过程的定义	过程的活动
名字的声明	名字的绑定
声明的作用域	绑定的生存期

- 静态概念
  - 在编译阶段可以确定。
  - 与程序文本结构有关。
- 动态概念
  - 在运行阶段才产生。
  - 与程序执行过程有关。

4. 活动记录

过程每次执行时，都需要一块连续的存储区。

- 含义
  - 这块存储区称为活动记录，也称为栈帧。
- 活动记录的组成
  - 临时数据
  - 局部数据
  - 保存的机器状态
  - 访问链
  - 控制链
  - 返回值
  - 参数
- 作用
  - 保存过程执行所需的信息。
  - 支持过程调用和过程返回。
  - 管理局部变量和参数。
  - 保存调用前后的控制信息。

5. 局部数据的布局

- 基本原则
  - 局部变量通常按照声明顺序分配空间。
  - 不同类型的数据占用空间不同。
  - 局部变量地址通常用相对地址表示。
- 数据对齐
  - 某些数据类型需要放在满足特定条件的地址上。
  - 数据对齐可以提高访问效率。
  - 由于对齐要求，局部数据之间可能出现空闲空间。
- 编译器的作用
  - 编译器可以在编译阶段确定大部分局部变量的位置。
  - 对于大小可变的数据，可能需要在运行时分配。

6. 程序块

- 含义
  - 程序块是含有局部声明的语句结构。
- 特点
  - 程序块可以嵌套。
  - 内层程序块可以声明与外层同名的变量。
  - 内层声明会遮蔽外层声明。
  - 不同程序块中的局部变量可以复用存储空间。
- 重点
  - 程序块决定局部变量的作用域。
  - 嵌套程序块会影响变量可见性。
  - 存储分配时可以利用程序块的作用范围复用空间。

6.2 全局栈式存储分配

1、运行时内存的划分

程序运行时的逻辑地址空间划分为四大区域，各区域特性与用途如下：

- **代码区**：存放目标程序代码，编译时确定长度，运行时不可修改，位于低地址空间。
- **静态数据区**：存储全局变量、常量、编译器辅助数据等，空间大小编译时确定，地址可直接编入目标代码，访问效率高。无递归的语言（如早期 FORTRAN）可将全部数据静态分配。
- **栈区**：存放过程的**活动记录**，用于管理过程调用与返回。空间动态变化，面向过程 / 面向对象语言普遍使用，适合生存期嵌套、有序回收的数据。
- **堆区**：由程序员手动分配、释放空间（如 C 语言malloc/free），数据生存期不受过程调用规则约束，独立管理。

栈与堆在地址空间两端**相向动态增长**；栈的分配回收开销远小于堆。

2、 活动树和运行栈

- **活动树**
  - 用来描述过程调用、执行、返回的控制流次序的树形结构：
  - 每个结点对应**一个过程活动**；根结点为主程序活动；
  - 父子结点代表调用关系（父活动调用子活动）；
  - 活动执行顺序对应活动树的**后根遍历。**
- **控制栈 & 运行栈**
  - **控制栈：**记录当前所有活跃（未执行完毕）的过程活动，活动开始入栈、结束出栈。
  - **运行栈：**将活动记录存入控制栈，即**活动记录栈。**过程调用时压入新活动记录，绑定局部变量；过程返回时弹出活动记录，局部变量失效。
  - **特点：**支持递归调用，过程每一次调用都会生成独立的活动记录。

参数、返回值放在活动记录头部，靠近调用者，便于参数传递，也支持可变参数（如printf）。

- 活动记录布局设计原则
  - 控制链、访问链、机器状态等定长数据放在活动记录中间。
  - 编译时大小不确定的数据（动态数组、临时变量）放在活动记录尾部。

3、 调用序列与返回序列

> 过程调用、返回需要配套代码管理运行栈、保存 / 恢复现场，分为调用序列和返回序列，代码由调用者和被调用者共同完成。

- 调用序列（调用者 → 被调用者）
  - 调用者计算实参，逆序压入栈，预留返回值空间；
  - 调用者压入返回地址，切换控制权到被调用过程；
  - 被调用者保存寄存器、机器状态，建立控制链（动态链）（指向调用者活动记录）；
  - 被调用者分配局部变量、临时数据空间，开始执行过程体。
- 返回序列（被调用者 → 调用者）
  - 被调用者将运行结果写入返回值区域；
  - 释放局部数据空间，恢复寄存器与机器状态；
  - 根据返回地址回到调用者；
  - 调用者回收参数占用的栈空间，读取返回值。

4、栈上可变长度数据

对于**编译时无法确定大小**的数据（如依赖参数的动态数组），处理方案：

- 在活动记录中预留**指针单元**，而非直接分配数组空间；
- 过程运行、确定数组实际大小后，在栈顶动态分配数组内存；
- 将数组起始地址存入预先预留的指针单元，后续通过指针间接访问数组元素。
- 该方式可规避堆管理的开销，仅适用于过程结束后不再使用的数据。

5、悬空引用

- **定义**
  - 引用了**已经被回收**的存储单元，该引用称为悬空引用，对应的指针为悬空指针。
- **产生原因**
  - 典型场景为过程返回指向局部变量的指针。过程结束后，其活动记录被栈回收，原局部变量空间失效。
- **危害**
  - 已回收空间可能被重新分配，导致程序出现无规律、难以排查的逻辑错误。栈和堆都可能出现该问题。

6、重难点

- 结合寄存器 `ebp` 、 `esp` ，理解栈帧（活动记录）在 x86 架构下的内存布局与寻址方式。
- 区分 **控制链（动态链）**的作用，理解过程嵌套调用时栈的联动关系。
- 理解可变参数函数（如 `printf` ）依托栈布局实现参数读取的原理。
- 动态数组在栈上的分配逻辑，区分静态局部变量与栈上动态数据的差异。
- 分析代码场景中悬空引用的形成过程。

5.5 类型表达式的等价

1. 基本概念

- 类型检查中常需要判断两个类型表达式是否相同。
- 如果两个类型表达式相同，则返回该类型。
- 如果两个类型表达式不同，则返回 `type_error` 。
- 类型表达式的等价主要用于判断表达式、变量、函数参数等是否类型匹配。

2. 类型名字带来的问题

- 在某些程序语言中，可以给类型表达式命名。
- 类型名字可以在后续类型表达式中继续使用。
- 类型名字的使用会带来歧义：
  - 相同名字是否一定表示相同类型?
  - 不同名字是否可能表示相同类型?
- 因此引出了两种类型等价概念：
  - 结构等价
  - 名字等价

3. 类型表达式的结构等价

- 含义
  - 结构等价是指两个类型表达式的结构完全相同。
- 判断规则
  - 如果两个类型表达式是相同的基本类型，则等价。
  - 如果两个类型表达式使用相同的类型构造符，则继续比较其组成部分。
  - 对数组、指针、函数等复合类型，需要递归比较内部类型。
  - 如果结构不一致，则不等价。
- 例子
  - `integer` 与 `integer` 结构等价。
  - `pointer(integer)` 与 `pointer(integer)` 结构等价。
  - `array(s1, s2)` 与 `array(t1, t2)` 是否等价，要继续判断 `s1` 与 `t1` 、 `s2` 与 `t2` 是否等价。

4. 类型表达式的名字等价

- 含义
  - 名字等价是指根据类型名字是否相同来判断类型是否等价。
- 判断规则
  - 同一个类型名字表示同一种类型。
  - 不同类型名字一般被看作不同类型。
  - 即使两个类型结构完全相同，如果名字不同，也可能不等价。
- 特点
  - 名字等价比结构等价更严格。
  - 名字等价可以避免递归类型中无限展开的问题。
  - 不同语言对名字等价和结构等价的处理规则可能不同。

5. 记录类型

- 含义
  - 记录类型是一种复合类型，由若干个字段组成。
- 特点
  - 每个字段都有字段名和对应类型。
  - 记录类型常用于描述结构体或记录数据。
  - 字段访问时需要检查：
    - 字段是否存在；
    - 字段类型是否正确。
- 相关规则
  - 定义记录类型时，要保存字段名和字段类型。
  - 访问记录字段时，要根据字段名查找对应类型。

6. 类型表示中的环

- 产生原因
  - 链表、树等递归数据结构会在类型表示中形成环。
  - 例如，一个 `cell` 结构体中包含指向 `cell` 的指针。
- 问题
  - 如果直接展开递归类型，可能导致无限递归。
  - 结构等价判断会变得复杂。
- 解决思路
  - 使用类型名表示递归类型。
  - 避免在类型图中无限展开。
  - 对记录类型通常采用名字等价，可以减少判断复杂度。